

Traffic Density and Intersection Monitoring System

Final Report

Alphan Algül
2584639

Emir Daregenli
2526226

CNG 476 – System Simulation

Spring 2025–2026

Instructor: Asst. Prof. Dr. Muhammad Toaha R. Khan

Group Number: 2

Github Repository: https://github.com/drgewr/Traffic_Density_and_Intersection_Monitoring_System

Date of Submission: 18th May 2026

Abstract—As the population increases, naturally the number of vehicles and pedestrians also increase, leading to a higher need for smart traffic monitoring systems for traffic safety and congestion management. We created our Traffic Density and Intersection Monitoring System to help solve this issue. We implemented four roadside sensors measuring different traffic events such as traffic density, pedestrian events and traffic accidents. Then the collected data is converted into packets and sent to the central management server to be processed. We used OMNeT++ as the main simulation tool implementing sensors, servers, randomly generating stochastic processes such as pedestrian event, traffic accidents etc. using distributions such as Poisson and Bernoulli while we used INET and FLoRa to facilitate the communication between the road sensors and the central management server. To analyze the performance of our system we used the performance metrics number of processed events, number of dropped events, end-to-end delay and the average server queue length. For the most part, the system behaved as we expected in the simulations apart from one strange case, being that scenario 2 having a shorter end-to-end delay compared to scenario 1 even though it has a shorter packet interarrival time. In theory, we would expect the opposite but repeated runs proved otherwise.

II. PROJECT DESCRIPTION

I. INTRODUCTION

As the population increases, naturally the number of pedestrians and vehicles also increase. This increases the need for smart transportation systems as traditional methods such as traffic police and camera monitoring may no longer be sufficient to deal with increased traffic. Therefore; by using IoT devices such as sensors, we can monitor traffic density, pedestrians, incidents and vehicle queues in real time for better traffic safety and congestion management. We used simulations for our smart traffic management system because real-life implementation is expensive and difficult and by using simulations, we can test the correctness of the system while saving time and resources. Our objective with this smart traffic monitoring system is to report metrics such as pedestrian events, incidents and vehicle queues to a central management system to be monitored for congestion management and pedestrian safety.

Traffic Density and Intersection Monitoring System models a real-life road with roadside sensors detecting traffic events such as vehicle queue lengths, traffic density, traffic accidents and pedestrian events developed using OMNeT++, INET and FLoRa. The purpose of the simulation is to model a smart traffic monitoring system that detects traffic events, send them to the main server to be processed and evaluated for traffic safety and congestion management.

Item	Description
Traffic Density and Intersection Monitoring System	Smart traffic monitoring system is simulated using OMNeT++, INET and FLoRa, using sensors to collect traffic data such as vehicle queue length, pedestrian events, traffic incidents and traffic density and then sent them to the main management server to be monitored
Application Domain	IoT based Traffic Management, Smart Cities, Smart Traffic Monitoring
Main Problem	As the population increases, naturally the number of pedestrians and vehicles also increase. This increases the need for smart transportation systems as traditional methods such as traffic police and camera monitoring may no longer be sufficient to deal with increased traffic.
Simulation Objective	Monitoring pedestrian events, number of vehicles on the road, vehicle queue buildup and congestion and traffic incidents for traffic safety and congestion management
Main Users/Entities	Pedestrians, Vehicles, Roadside Sensors, Traffic Management Server
Main Performance Metrics	Processed events, dropped events, end-to-end delay, server queue length

TABLE I
TRAFFIC DENSITY AND INTERSECTION MONITORING SYSTEM SUMMARY

III. SYSTEM ARCHITECTURE

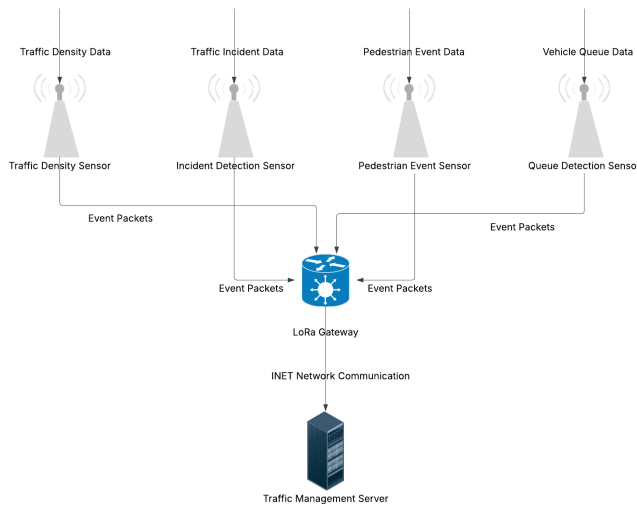


Fig. 1. System Architecture of Traffic Density and Intersection Monitoring System

Our project consists of four sensors Traffic Density Sensor, Incident Detection Sensor, Pedestrian Event Sensor and Queue Detection Sensor. These sensors collect data from the traffic such as pedestrian events, vehicle queue length etc. and send them to the Traffic Management Server as packets via LoRa gateway using INET and FLoRa frameworks.

IV. SYSTEM MODEL AND ASSUMPTIONS

Our system has 5 main entities, the four roadside sensors: Traffic Density Sensor, Incident Detection Sensor, Pedestrian Event Sensor and Queue Detection Sensor and Traffic Management Server. The roadside sensors detect their respective events (traffic density, pedestrian event, vehicle queue length) etc., update their state variables based on detections and send them as packets to the Traffic Management Server via wireless communication using FLoRa and INET. The main queueing point of the system is at the Traffic Management Server. Packets coming from the sensors are added to the server queue before being processed, if the queue is full they are dropped. Traffic density (number of vehicles), vehicle queue length and pedestrian events are generated using Poisson distribution while sensor failures and traffic accidents are generated using Bernoulli distribution.

Entity	Events	State Variables
Traffic Density Sensor	Detecting the number of vehicles and traffic density, sending traffic density and number of vehicles data as packets	Number of vehicles, Traffic density, Sensor status
Incident Detection Sensor	Detecting whether an accident occurred or not, sending accident data as packets	Accident status, Number of vehicles, Sensor status
Pedestrian Event Sensor	Detecting pedestrian events (movement, arrival etc.), sending pedestrian event data as packets	Number of pedestrians, Sensor status
Queue Detection Sensor	Detecting the length of the vehicle queue, sending vehicle queue data as packets	Number of vehicles, Vehicle queue length, Sensor status
Traffic Management Server	Packets arriving, dropping a packet, processing a packet	Processed events, Dropped events, Server queue length, End-to-End delay

TABLE II
SIMULATION ENTITIES, EVENTS, AND STATE VARIABLES

Assumption	Justification
M/M/1 Modelling	We modeled our queue and server with M/M/1 modelling with a single server and queue to reduce simulation complexity and computational cost
Traffic events are generated randomly	We modeled traffic events such as traffic density, length of vehicle queue, pedestrian events, sensor failures and traffic accidents as being generated randomly to model real life as none of these events are deterministic, they occur randomly
Stationary Sensors	In real life roadside sensors are usually set up in fixed positions so we implemented as such
Simple Wireless Communication	All sensors use the same communication behavior (generating packets, transmitting them wirelessly to the same server using FLoRa and INET). We implemented our sensors as such to simplify our simulation as detailed FLoRa modelling is outside of the scope of our project

TABLE III
MODELING ASSUMPTIONS AND JUSTIFICATION

V. SIMULATION METHODOLOGY

We modeled our simulation Traffic Density and Intersection Monitoring System as a stochastic discrete-event simulation and no time-stepped or trace-driven aspects were considered. Traffic events such as vehicle queue length, traffic density (number of vehicles), pedestrian events such as movement and arrival, sensor failures and traffic accidents are all modeled as stochastic processes generated as random numbers with distributions such as Poisson and Bernoulli.

Our simulation is considered as a discrete-event simulation because the system state changes based on discrete events rather than with time such as a packet being generated from the collected data sent to the main server or when a packet arrives at the main server if the server queue is full, the packet is dropped and the metric dropped events is updated

For Monte Carlo repeated runs, we used the built-in feature of OMNeT++. OMNeT++ automatically changes the seed for every run so we can manually run the application multiple times to compare metrics such as processed events, dropped events, end-to-end delay etc. for Monte Carlo repeated runs

Event-scheduling generation logic is handled via self messages and the main server. Self messages generated with exponential distribution schedule when sensors detect traffic events and generate packets. The main server schedules when packet processing starts or finishes using packet processing times generated using exponential distribution.

Process	Distribution/Model	Purpose in the Simulation
Traffic Density Generation	Poisson Distribution	To model the number of vehicles on the road. This data will be detected by the Traffic Density Sensor.
Vehicle Queue Length Generation	Poisson Distribution	To model the number of vehicles waiting in the queue. This data will be detected by the Queue Detection Sensor
Pedestrian Event Generation	Poisson Distribution	To model pedestrian events such as movement and arrival. This data will be detected by the Pedestrian Event Sensor
Sensor Failure Generation	Bernoulli Distribution	To model roadside sensor failures
Traffic Accident Generation	Bernoulli Distribution	To model the traffic accidents on the road
Event Interarrival Generation	Exponential Distribution	To model intervals between the sensors detecting events
Server Processing Time Generation	Exponential Distribution	To model the processing time for each packet that arrive at the server

TABLE IV
RANDOM PROCESSES AND PROBABILITY MODELS

Scheduled Event	Triggered Condition	State Change/Action
Traffic Event Packet Generation	Self message indicating it is time to detect traffic event and generate a packet arriving	Traffic event packet is created and sent to the main server
Packet Arrival and Drop	Traffic event packet arrives at the server	If the server queue is not full, packet is added to the queue. If it is full, the packet is dropped and dropped events metric is updated
Server Finishing Processing a Packet	Packet processed for a randomly generated service time	Server starts processing next packet

TABLE V
SCHEDULED EVENTS IN DISCRETE-EVENT SIMULATION

VI. OMNeT++, INET, AND FLoRa IMPLEMENTATION

For the implementation of our project, we used OMNeT++ as the main simulation engine. We created 4 road side sensors: Sensor 0 being the Traffic Density Sensor detecting the number of vehicles on the road, Sensor 1 being the Incident Detection Sensor, detecting traffic accidents, Sensor 2 being

the Pedestrian Event Sensor, detecting pedestrian arrivals and movement and Sensor 3 being the Queue Detection Sensor detecting vehicle queue build up and congestion. Then the collected data are sent as packets to the main server using INET and FLoRa.

File Name	File Type	Purpose
package.ned	NED	Defines the package structure of the project
TrafficSensors.ned	NED	Defines interarrival times, average vehicle and pedestrian counts, vehicle queue length, and parameters for accidents and sensor faults
TrafficServer.ned	NED	Defines server service time, queue capacity, signal types (queue length, processed events etc.), statistics from these signals and traffic management server input gates
TrafficNetwork.ned	NED	Defines sensors, their id, type (traffic density sensor, pedestrian event sensor etc.) and the connection between sensors and server
TrafficSensors.cc	C++	Generates vehicle counts, pedestrian events and vehicle queue lengths using Poisson distribution and sensor failures and traffic accidents using Bernoulli distribution, collects these data using sensors and sends them to the main server
TrafficServer.cc	C++	Receives data (sensor failure, pedestrian arrival etc.) and adds it to the event queue (if the queue is full, it is dropped). Then it calculates metrics such as number of processed events and end-to-end delays
TrafficSensors.h	Header	Defines structure and methods for TrafficSensors.cc such as event generation and signal generation
TrafficServer.h	Header	Defines the structure and methods for TrafficServer.cc such as handling incoming sensor data, processing events, and calculating statistics such as count of dropped/processed events and end-to-end delay
omnetpp.ini	INI	Configures OMNeT++, enables scalar and vector stats for analyzing performance and runs the simulation for 5000 seconds and 10 repeats to analyze the confidence interval accurately
TrafficEvent.msg	MSG	Defines a packet structure TrafficEvent which facilitates communication between sensors and server and stores data such as (sensor id, event type, vehicle count, accident status, traffic density etc.)

TABLE VI
IMPLEMENTED OR MODIFIED PROJECT FILES

Tool/Framework	How It Is Used in the Project
OMNeT++	We used OMNeT++ as the main engine for simulating the project. It runs the discrete-event simulation, manages simulation time and calculates statistics
INET	We used INET for establishing the network communication between the roadside sensors and the main server. Roadside sensors read data and send them as packets via INET to the main server
FLoRa	We used FLoRa to simulate the wireless environment and transmission between the 4 roadside sensors and the main server

TABLE VII
OMNET++, INET, AND FLOrA USAGE

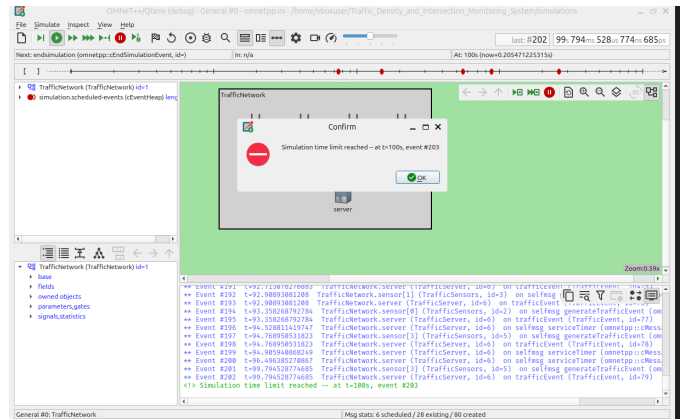


Fig. 4. Simulation terminating itself after 100 seconds

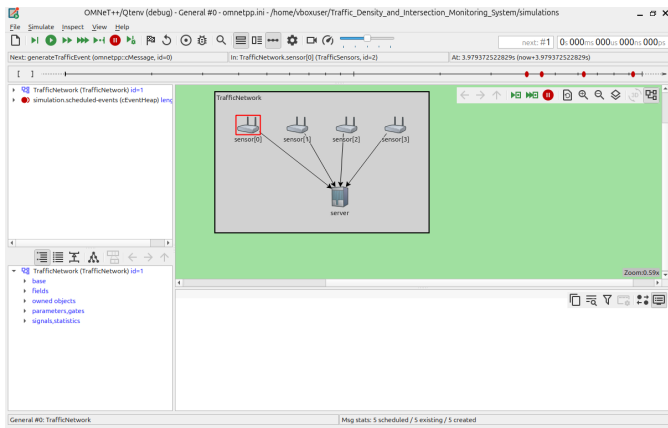


Fig. 2. Four Roadside Sensors and Main Server

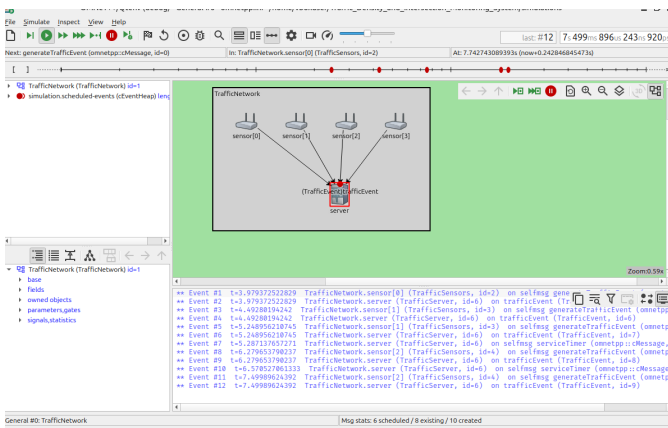


Fig. 3. Event Generation and Communication between Sensors and Server

VII. MAPPING OF COURSE TOPICS TO THE PROJECT

Course Topic	Project-Specific Implementation
Simulation methodology and modelling	We modeled our system as having 4 roadside sensors and a central management server. We assumed that the system has M/M/1 modelling, traffic events are generated randomly and the sensors are stationary. Our goal with this system is to build a smart traffic management system for efficient traffic and congestion management
Random number generation	Pseudo-random numbers are used throughout the simulation whenever there is a value generated randomly such as number of vehicles or sensor failures
Monte Carlo simulation	We used the built-in random seed generation feature of OMNeT++ to run our project many times and measure metrics such as average end-to-end delay, server queue length etc.
Probability models	Traffic events in real life are random and unpredictable. To accurately simulate this, we used Poisson distribution for traffic density, pedestrian event and length of the vehicle queue generation, Bernoulli distribution for traffic accident and sensor failure generation and Exponential distribution for packet interarrival and processing time generation
Inverse transformation or sampling	We used OMNeT++'s built-in non-uniform sampling methods such as bernoulli(), poisson() and exponential() for generating traffic events and implementing scheduling behavior.
Stochastic processes	We implemented our simulation mainly as a combination of stochastic processes generated using distributions such as Poisson, Bernoulli and Exponential to model the unpredictability of real world traffic events.
Queueing analysis	The main queueing point of our simulation is at the main server. After being generated by roadside sensors using the data collected, traffic event packets are sent to the main server. If server queue is full, they are dropped. If the server queue is not full, they are added to the server queue and wait to be processed.
Discrete-event simulation	We modeled our project as a discrete event simulation because the simulation state only changes with traffic events, packet arrivals and packet completion times.
Event scheduling	Our simulation uses Exponential Distribution to schedule events such as packet interarrival and processing times.
Time-stepped / trace-driven view	The simulation is mainly developed as a stochastic discrete event simulation. We haven't considered time-stepped and trace-driven views for simplification
Wireless network modelling	We implemented wireless networking using FLoRa with traffic event packets transmitted from the roadside sensors to the main server wirelessly
OMNeT++ implementation	We have used OMNeT++ as the main simulation engine for our system. We used OMNeT++'s built-in random seeds for Monte Carlo repeated runs, methods such as bernoulli(), exponential() and poisson() for stochastic process generation and graph support to generate box plots and vector graphs for output analysis
INET integration	We used INET for packet handling logic between the roadside sensors and the main server
FLoRa integration	We used FLoRa to facilitate the wireless communication between the roadside sensors and the main traffic management server
Input analysis	We justified input parameters such as sensor failures, traffic density, packet interarrival time etc. mathematically using Poisson, Exponential and Bernoulli distributions
Output analysis	We have analyzed the outputs of 2 scenarios with first scenario having slower packet interarrival time (exponential(105s)) while the second scenario having a faster packet interarrival time (exponential(50s)). We have used vector graphs, box plots, confidence intervals and different performance metrics such as end-to-end delay and server queue length to compare and analyze these two scenarios
IoT application relevance	This simulation is very relevant to IoT as it is a smart traffic management system. It uses IoT devices such as roadside sensors that detect traffic events, sends them to be processed at a main server and measures metrics such as end-to-end delay and processed events to evaluate system performance

TABLE VIII

MAPPING OF CNG 476 COURSE TOPICS TO THE FINAL PROJECT

VIII. EXPERIMENTAL DESIGN

We tested our simulation with 2 different scenarios. Scenario 2 having a shorter packet interarrival time (exponential(50s)) while scenario 1 having a longer interarrival time (exponential(105s)). We compared the results of these scenarios by comparing metrics such as end-to-end delay and server queue lengths.

Item	Value/Description
Simulation Duration	5000 seconds
Number of Runs	10 (5 for each scenario)
Random Seeds	Randomly Generated using OMNeT++'s built-in random seed generation feature
Baseline Scenario	Packet interarrival time generated with Exponential distribution exponential(105s) (longer packet interarrival)
Alternative Scenario	Packet interarrival time generated with Exponential distribution exponential(50s) (shorter packet interarrival)
Varied Parameters	Packet interarrival time. Scenario 2 has shorter packet interarrival time while scenario 1 has longer interarrival time.
Collected Metrics	Processed events, Dropped events, End-to-end delay, Server queue length

TABLE IX

EXPERIMENTAL DESIGN SUMMARY

IX. PERFORMANCE METRICS

Metric	Unit	Definition/Purpose
Processed Events	number of events	To measure whether the server successfully receives traffic event packets and processes them or not to evaluate server performance
Dropped Events	number of events	To measure how many traffic events are dropped by the server because the server queue is full to evaluate server performance
Server Queue Length	mean number of packets in server queue	To measure the mean server queue length to evaluate how efficient is the server at processing traffic events
End-to-End Delay	ms	To measure the total time between a traffic event packets generation in the roadside sensor to its processing being completed to evaluate how fast the server is

TABLE X
PERFORMANCE METRICS

X. INPUT ANALYSIS

In the real world, traffic events such as accidents, pedestrian event and sensor failures happen randomly while number of vehicles, pedestrians and the length of the queue of vehicles are ever changing. Sensors detect traffic events, generate packets and send these packets to a server and these packets may have different processing times based on their content. In order to model the randomness and unpredictability of the traffic events, we used Poisson, Bernoulli and Exponential distributions.

Input Variable		Analysis Method	Conclusion
Traffic (Number of Vehicles)	Density of	Poisson Distribution	Number of Vehicles randomly change all the time in real world with vehicles entering and leaving certain sections of the road. In order to model this behavior, we used Poisson distribution to randomly generate the number of vehicles per traffic event.
Pedestrian Events		Poisson Distribution	Pedestrian events such as arrival and movement happen randomly in real life for example in one instance there are 15 pedestrians in an intersection while in other there are 25 pedestrians. In order to model this behavior we used Poisson distribution to randomly generate pedestrian events.
Number of Vehicles in Vehicle Queue		Poisson Distribution	Number of Vehicles in a Queue is random as certain number of cars arrive and leave the intersection at a time. For example there could be 15 cars waiting for green light while at other times it could be 100 cars. To model this behavior, we used Poisson distribution.
Sensor Failures		Bernoulli Distribution	Roadside sensors might fail at any time. We modeled this behavior using Bernoulli distribution to generate sensor failures
Traffic Accidents		Bernoulli Distribution	Traffic accidents happen randomly and at any time in the real world. We modeled this behavior using Bernoulli distribution to generate traffic accidents
Traffic Event Packet Interarrival Times		Exponential Distribution	Traffic event packets are generated randomly by sensors independent of each other and sent to the main server and in order to model this, we used exponential distribution
Packet Processing Times		Exponential Distribution	Packets that arrive at the main server may have different processing times. Some packets might contain a lot of information while other packets might contain very little information and are easily processed. To model different kinds of packets we used exponential distribution to generate a random processing time for each packet

TABLE XI
INPUT ANALYSIS SUMMARY

XI. RESULTS AND OUTPUT ANALYSIS

With Scenario 1 having a longer packet interarrival time while scenario 2 having a shorter packet interarrival time we expect scenario 1 to have less processed and dropped packets and less average server queue length. One interesting observation is that we expect the end to end delay to be lower for scenario 1 as it has shorter interarrival times but actually it is the opposite

Scenario	Metric	Average	Standard Deviation/Confidence Interval
Scenario 1 (Base Scenario)	Processed Events	47	0
Scenario 1 (Base Scenario)	Dropped Events	134	0
Scenario 1 (Base Scenario)	Server Queue Length	8.457944	2.245668
Scenario 1 (Base Scenario)	End-to-End Delay	940.557229	352.386101
Scenario 2 (Base Scenario)	Processed Events	55	0
Scenario 2 (Base Scenario)	Dropped Events	292	0
Scenario 2 (Base Scenario)	Server Queue Length	8.788618	2.065533
Scenario 2 (Base Scenario)	End-to-End Delay	866.124872	366.220118

TABLE XII
SIMULATION RESULTS SUMMARY

▼ TrafficNetwork.server	
> droppedEvents:count (scalar)	134
> endToEndDelay:mean (scalar)	940.557229
> endToEndDelay:stats (statistics)	940.557229 (47)
> endToEndDelay:vector (vector)	940.557229 (47)
> processedEvents:count (scalar)	47
> queueCapacity (parameter)	10
> queueLength:max (scalar)	10
> queueLength:mean (scalar)	8.457944
> queueLength:vector (vector)	8.457944 (107)
> serviceTime (parameter)	exponential(105s)
> typename (parameter)	"trafficedensityandintersectionmonitoringsystem.TrafficServer"

Fig. 5. Scenario 1 Metrics

▼ processedEvents:stats (statistics)	1 (47)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	47
Sum of weights	47
Mean	1
StdDev	0
Min	1
Max	1
Recordingmode	stats
Source	processedEvents
Title	Processed Events

Fig. 6. Scenario 1 Processed Events Detailed Output

▼ droppedEvents:stats (statistics)	1 (134)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	134
Sum of weights	134
Mean	1
StdDev	0
Min	1
Max	1
Recordingmode	stats
Source	droppedEvents
Title	Dropped Events

Fig. 7. Scenario 1 Dropped Events Detailed Output

▼ queueLength:stats (statistics)	8.457944 (107)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	107
Sum of weights	107
Mean	8.457944
StdDev	2.245668
Min	0
Max	10
Recordingmode	stats
Source	queueLength
Title	Queue Length

Fig. 8. Scenario 1 Server Queue Length Detailed Output

▼ endToEndDelay:stats (statistics)	940.557229 (47)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	47
Sum of weights	47
Mean	940.557229
StdDev	352.386101
Min	57.860091
Max	1,369.211735
Recordingmode	stats
Source	endToEndDelay
Title	End-to-End Delay

Fig. 9. Scenario 1 End-to-End Delay Detailed Output

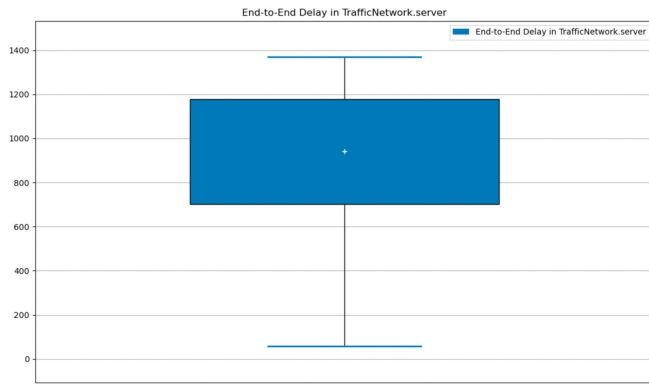


Fig. 10. Scenario 1 Box Plot showing the distribution of end-to-end delay values

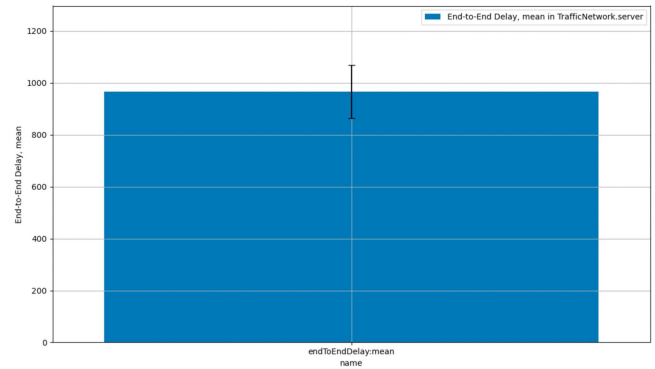


Fig. 13. Scenario 1 Confidence Interval Graph for mean end-to-end delay (The blue bar represents the mean end-to-end delay, while the black vertical bars show the 95 percent confidence interval across repeated simulation runs)

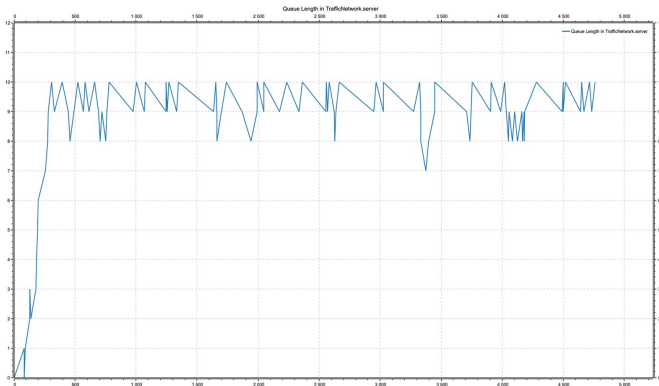


Fig. 11. Scenario 1 values of server queue length throughout the simulation

TrafficNetwork.server	
droppedEvents:count (scalar)	292
droppedEvents:mean (scalar)	1
droppedEvents:stats (statistics)	1 (292)
endToEndDelay:mean (scalar)	866.124872
endToEndDelay:stats (statistics)	866.124872 (55)
endToEndDelay:vector (vector)	866.124872 (55)
processedEvents:count (scalar)	55
processedEvents:mean (scalar)	1
processedEvents:stats (statistics)	1 (55)
queueCapacity (parameter)	10
queueLength:mean (scalar)	8.788618
queueLength:stats (statistics)	8.788618 (123)
queueLength:vector (vector)	8.788618 (123)
serviceTime (parameter)	exponential(105s)
typename (parameter)	"trafficdensityandintersectionmonitoringsystem.TrafficServer"

Fig. 14. Scenario 2 Metrics

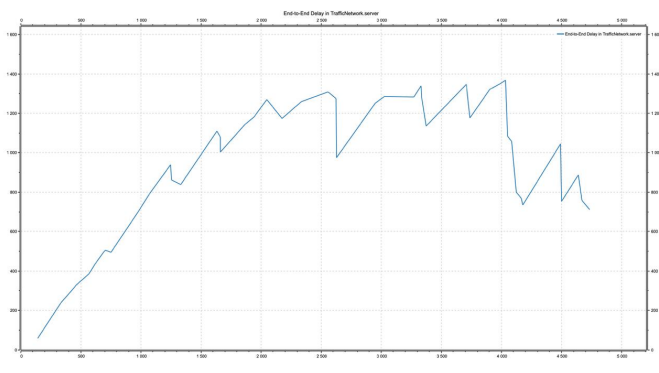


Fig. 12. Scenario 1 values of end-to-end delay throughout the simulation

processedEvents:stats (statistics)		1 (55)
Module name	TrafficNetwork.server	
Type	double	
Kind	unweighted	
Count	55	
Sum of weights	55	
Mean	1	
StdDev	0	
Min	1	
Max	1	
Recordingmode	stats	
Source	processedEvents	
Title	Processed Events	

Fig. 15. Scenario 2 Processed Events Detailed Output

droppedEvents:stats (statistics)	1 (292)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	292
Sum of weights	292
Mean	1
StdDev	0
Min	1
Max	1
Recordingmode	stats
Source	droppedEvents
Title	Dropped Events

Fig. 16. Scenario 2 Dropped Events Detailed Out

queueLength:stats (statistics)	8.788618 (123)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	123
Sum of weights	123
Mean	8.788618
StdDev	2.065533
Min	0
Max	10
Recordingmode	stats
Source	queueLength
Title	Queue Length

Fig. 17. Scenario 2 Server Queue Length Detailed Output

endToEndDelay:stats (statistics)	866.124872 (55)
Module name	TrafficNetwork.server
Type	double
Kind	unweighted
Count	55
Sum of weights	55
Mean	866.124872
StdDev	366.220118
Min	57.860091
Max	1,422.595692
Recordingmode	stats
Source	endToEndDelay
Title	End-to-End Delay

Fig. 18. Scenario 2 End-to-End Delay Detailed Output

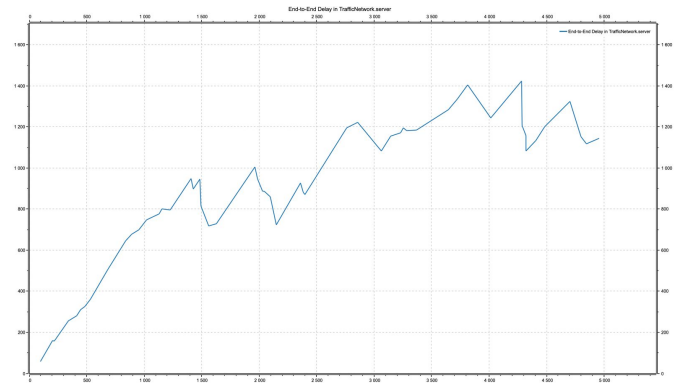


Fig. 19. Scenario 2 values of end-to-end delay throughout the simulation

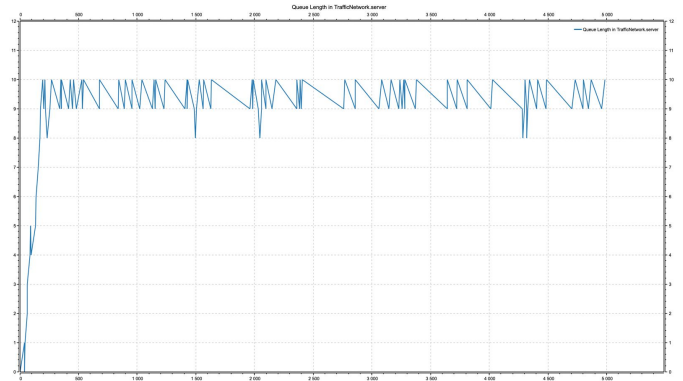


Fig. 20. Scenario 2 values of server queue length throughout the simulation

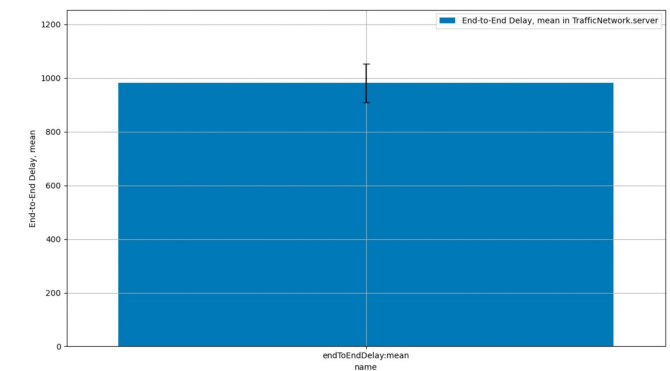


Fig. 21. Scenario 2 Confidence Interval Graph for mean end-to-end delay

XII. DISCUSSION

The simulation behaved as expected for the most part for the two scenarios with scenario 1 having less processed and dropped packets and less average server queue length. The unexpected behavior was scenario 2 having a shorter end-to-end delay than scenario one but we expected the opposite as scenario 1 has a longer interarrival time.

Scenario 2 with shorter interarrival time allows us to collect more traffic data from the roadside sensors, increasing the number of processed events, it also increases the server queue length which leads to more dropped events.

The main limitation of the system is that it simplifies the wireless communication between the roadside sensors and the main server. We implemented the system as such to reduce complexity making development faster but at the cost of not being able to simulate realistic wireless communications.

Finding	Explanation
Number of processed and dropped packets and the average length of the server queue increases for Scenario 2	This behavior is expected since scenario 2 has a shorter interarrival time so it receives more packets, so its queue is on average longer while dropping more events
Shorter end-to-end delay for Scenario 2	This is an unexpected behavior by the system. In theory, scenario 1 should have a shorter end-to-end delay as it has longer packet interarrival time but the runs prove otherwise.

TABLE XIII
DISCUSSION OF MAIN FINDINGS

XIII. CONCLUSION

In conclusion, we implemented our project Traffic Density and Intersection Monitoring System using four roadside sensors and a central server. We compared two scenarios one with shorter packet interarrival time and the other having a longer packet interarrival time. We used metrics such as end-to-end delay and average server queue length to compare and analyze the two scenarios. The simulation behaved as expected for the most part with the only unexpected behavior being that scenario 2 having a shorter end-to-end delay compared to scenario 1 even though we expect the opposite. The main limitation of the project is the fact that wireless communication between the sensors and the main server is greatly simplified to allow for faster development but in return not being able to model realistic wireless communications. One future improvement that could be made to the project is to further investigate the unexpected behavior of scenario 2 regarding end-to-end delays. Another improvement might be trying to reduce the number of dropped packets to not waste data collected by the sensors

XIV. CODE AVAILABILITY

Project source code, simulation and configuration files, scripts and final report pdf are publicly available at: https://github.com/drgewr/Traffic_Density_and_Intersection_Monitoring_System

XV. INDIVIDUAL CONTRIBUTIONS

Name	Student ID	Contribution
Alphan Algül	2584639	Worked mainly on literature research and report writing.
Emir Daregenli	2526226	Worked mainly on the implementation of the simulation using OMNeT++, INET and FLoRa.

TABLE XIV
INDIVIDUAL CONTRIBUTIONS

REFERENCES

- [1] A. K. Khlaif, M. S. Croock, and S. H. Shaker, "Simulating Traffic Lights Control using Wireless Sensor Networks," *International Journal of Computer Applications*, vol. 104, no. 12, pp. 1–8, 2014. <https://research.ijcaonline.org/volume104/number12/pxc3899445.pdf>
- [2] M. Collotta and G. Pau, "New Solutions Based on Wireless Networks for Dynamic Traffic Lights Management," *Transport and Telecommunication Journal*, vol. 16, no. 4, pp. 269–279, 2015. <https://reference-global.com/download/article/10.1515/tjt-2015-0021.pdf>
- [3] S. Faye, C. Chaudet, and I. Demeure, "Influence of Radio Communications on Multiple Intersection Control by a Wireless Sensor Network," in *Proceedings of the 13th International IEEE Conference on Intelligent Transportation Systems Telecommunications*, 2013. https://www.researchgate.net/publication/261302914_Influence_of_radio_communication
- [4] S. Faye and C. Chaudet, "Characterizing the Topology of an Urban Wireless Sensor Network for Road Traffic Management," *IEEE Transactions on Vehicular Technology*, 2016. https://www.researchgate.net/publication/282527317_Characterizing_the_Topology_of_
- [5] A. A. Alhaj et al., "Improving the Smart Cities Traffic Management Systems using VANETs and IoT Features," *Journal of Statistics Applications & Probability*, vol. 12, no. 2, pp. 405–414, 2023. <https://www.naturalspublishing.com/files/published/u869yhu861b238.pdf>
- [6] T. Ormándi et al., "The Importance of V2X Simulation: An In-depth Comparison of Intersection Control Algorithms," *Simulation Modelling Practice and Theory*, 2023. <https://www.sciencedirect.com/science/article/pii/S2214209623001067>
- [7] C. Sommer, L. Leontiadis, R. German, and F. Dressler, "A Networking Perspective on Self-Organizing Intersection Management," in *Proceedings of the IEEE World Forum on Internet of Things*, 2014. <https://www.cms-labs.org/bib/sommer2014networking/sommer2014networking.pdf>
- [8] T. Petrov, A. Kesting, and A. Toledo, "A Framework Coupling VISSIM and OMNeT++ to Simulate Future Intelligent Transportation Systems," 2021. https://toledo.net.technion.ac.il/files/2022/01/Communications_simulation_21.pdf
- [9] J. Contreras et al., "An Algorithm Based on VANET Technology to Count Vehicles," 2020. https://digitalcommons.jsu.edu/cgi/viewcontent.cgi?article=1129&context=fac_res
- [10] N. Cárdenas-Benítez et al., "Traffic Congestion Detection System through Connected Vehicles and Big Data," 2016. https://www.scienceopen.com/document_file/aae47f80-8a44-4348-9038-7a9e4b5e3875/PubMedCentral/aae47f80-8a44-4348-9038-7a9e4b5e3875.pdf
- [11] Beenish et al., "A Novel Markov Model-Based Traffic Density Estimation Technique," 2023. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9866053/>
- [12] C. Sommer, D. Eckhoff, R. German, and F. Dressler, "Veins: The Open Source Vehicular Network Simulation Framework," 2019. <https://disi.unitn.it/~msegata/bibfiles/sommer2019veins.pdf>
- [13] K. Shafiee et al., "Modeling and Simulation of Vehicular Networks," 2011. <https://people.ece.ubc.ca/~vleung/NSERC-InvitedTalks/20Papers/divan12c-shafiee.pdf>

- [14] M. Slabicki, G. Premsankar, and M. Di Francesco, "Adaptive Configuration of LoRa Networks for Dense IoT Deployments," in *Proceedings of the IEEE/IFIP Network Operations and Management Symposium*, 2018.
<https://flora.aalto.fi/resources/slabicki-2018-noms.pdf>
- [15] L. Leonardi et al., "MRT-LoRa: A Multi-hop Real-time Communication Protocol for Industrial IoT," *Computer Communications*, 2023.
<https://www.sciencedirect.com/science/article/abs/pii/S0140366422004637>